

QwenSwarm: RepoPilot

Zero-Copy Multi-Agent Software Engineering System

Agent Society Track — Qwen Cloud Global AI Hackathon

Overview

RepoPilot is an autonomous software engineering system designed to understand, modify, validate, and repair large codebases using a swarm of specialized AI agents.

Unlike traditional coding agents that repeatedly reload repository context on every interaction, RepoPilot introduces a **Zero-Copy KV-Cache Architecture** powered by SGLang RadixAttention.

The repository is parsed once, transformed into a structured repository context, loaded into GPU memory, and reused across all agent interactions.

This dramatically reduces:

- latency
- repeated computation
- token waste
- repository reprocessing

while enabling scalable multi-agent collaboration.

The Problem

Modern coding agents face a hidden bottleneck.

Consider a repository containing:

- 200 files
- 50,000+ lines of code
- 20,000+ tokens of repository context

A typical workflow may require:

- planning
- implementation
- review

- debugging
- repair

Each step often reprocesses the same repository context.

Traditional flow:

```
Repository Context
  ↓
Agent Call

Repository Context
  ↓
Agent Call

Repository Context
  ↓
Agent Call
```

As repositories grow, latency and cost increase dramatically.

The system repeatedly pays the "token tax" for understanding code it already saw.

Our Solution

RepoPilot treats repository understanding as a shared memory resource.

The repository context is:

1. Parsed once.
2. Loaded into GPU memory.
3. Cached using SGLang RadixAttention.
4. Shared across all agents.

This transforms the workflow into:

```
Repository Context
  ↓
Cached Once
  ↓

Planner
Executor
```

Reviewer
Repair

Shared KV Cache

Instead of recomputing repository understanding, agents branch from the same cached prefix.

Core Innovation

Zero-Copy Swarm Architecture

The central innovation is a shared repository context that remains resident in GPU memory.

Every agent request begins with an identical prefix:

```
[  
  {  
    "role": "system",  
    "content": SHARED_SYSTEM_PROMPT  
  },  
  {  
    "role": "user",  
    "content": REPO_CONTEXT_PAYLOAD  
  }  
]
```

This enables:

- Prefix sharing
 - RadixAttention cache reuse
 - Near-zero repeated repository processing
-

Architecture

High-Level Workflow

flowchart TB

User[User Request]

```
User --> Planner

Planner --> ExecutorA
Planner --> ExecutorB
Planner --> ExecutorC

ExecutorA --> Sandbox
ExecutorB --> Sandbox
ExecutorC --> Sandbox

Sandbox --> Reviewer

Reviewer -->|Approved| Patch

Reviewer -->|Rejected| Repair

Repair --> Sandbox

Sandbox --> Reviewer

Patch --> User
```

Zero-Copy Inference Architecture

```
flowchart LR
    Repo[Repository]
    Repo --> ContextManager
    ContextManager --> SharedPrefix
    SharedPrefix --> SGLang
    SGLang --> RadixAttention
    RadixAttention --> Planner
    RadixAttention --> ExecutorA
    RadixAttention --> ExecutorB
    RadixAttention --> ExecutorC
    RadixAttention --> Reviewer
```

RadixAttention --> Repair

Repository Intelligence Layer

RepoPilot does not rely on raw file dumps.

Instead, repositories are transformed into a structured semantic representation.

The Context Manager constructs:

- Repository Summary
- Symbol Graph
- Dependency Graph
- Call Graph
- Documentation Context

This allows agents to retrieve meaningful execution neighborhoods instead of blindly consuming files.

Swarm Agents

Planner Agent

Responsibilities:

- Analyze user requests
- Identify affected modules
- Create execution plans
- Split work into tasks

Output:

Structured plan schema.

Executor Agents

Responsibilities:

- Modify source code
- Implement features

- Create patches
- Update tests

Multiple executors may run concurrently.

Reviewer Agent

Responsibilities:

- Analyze patches
- Inspect execution evidence
- Verify correctness
- Approve or reject changes

Reviewer never modifies code.

Repair Agent

Responsibilities:

- Analyze failures
- Fix broken implementations
- Retry execution
- Produce corrected patches

The repair cycle continues until:

- success
 - retry budget exhausted
-

Sandbox Execution Layer

RepoPilot includes a dedicated execution environment.

The sandbox provides:

- Python execution
- Pytest execution
- Ruff linting
- Mypy validation
- Patch verification

This allows the swarm to validate its own work.

Structured Outputs

All agents use schema-constrained outputs.

Technologies:

- Pydantic v2
- JSON Schema
- OpenAI response_format
- SGLang XGrammar

Benefits:

- deterministic parsing
- reliable orchestration
- reduced hallucinations
- safer agent communication

No regex-based parsing is used.

Technology Stack

Infrastructure

- Modal
 - A10G GPU
-

Inference

- SGLang
- RadixAttention

Future-compatible:

- vLLM
-

Foundation Model

Default:

Qwen2.5-7B-Instruct

Architecture is model-agnostic.

Future upgrades may include larger coding-focused models.

Repository Analysis

- tree-sitter
 - ast
 - networkx
-

Runtime

- Python 3.11+
 - asyncio
-

Validation

- pytest
 - mypy
 - ruff
-

Why Not LangChain?

RepoPilot intentionally avoids heavyweight orchestration frameworks.

Not used:

- LangChain
- CrewAI
- AutoGen
- LlamaIndex

Reason:

The core innovation is GPU-aware orchestration.

A lightweight asyncio-based swarm provides:

- lower overhead
 - better observability
 - cache-aware scheduling
 - greater architectural control
-

Planned Features

Semantic Code Graph

Repository knowledge graph built from:

- symbols
 - imports
 - call relationships
 - inheritance trees
-

Self-Healing Repair Loops

Automatic:

Code Generation → Validation → Failure Analysis → Repair → Retest

KV-Affinity Scheduler

Tasks will be grouped by repository locality to maximize cache reuse.

Example:

```
auth/*
```

and

```
billing/*
```

are processed separately to improve RadixAttention efficiency.

Speculative Executors

Multiple executors explore alternative implementations simultaneously.

Reviewer selects the best candidate.

Benchmarking

RepoPilot will measure:

- Time-To-First-Token
- Total Latency
- Token Usage
- Cache Hit Rate
- Throughput

Benchmarks compare:

Baseline

Repeated repository processing.

RepoPilot

Shared repository context.

Expected improvements:

- lower latency
 - lower token consumption
 - higher agent throughput
-

Repository Structure

```
qwenswarm-repopilot/
```

```
README.md
```

```
AGENT.md
```

```
prompts/
```

```
app/  
tests/  
docs/  
data/
```

Detailed structure is defined in AGENT.md.

Roadmap

Phase 1

Infrastructure

- Modal
- SGLang
- Qwen

Phase 2

Repository Intelligence

- Tree-Sitter
- Graph Construction
- Context Manager

Phase 3

Swarm Orchestration

- Planner
- Executor
- Reviewer

Phase 4

Sandbox Validation

- Execution Runtime
- Testing

- Evidence Collection
-

Phase 5

Self-Healing

- Repair Agent
 - Retry Loops
-

Phase 6

Performance Optimization

- KV Affinity Scheduling
 - Dynamic Batching
 - Benchmarking
-

Vision

RepoPilot explores a new paradigm for software engineering agents.

Rather than repeatedly reconstructing repository understanding, the system treats repository knowledge as a persistent memory structure shared across an entire swarm.

The result is a faster, more scalable, and more efficient approach to repository-scale autonomous software engineering.

Built for the Qwen Cloud Global AI Hackathon.

Agent Society Track.

Powered by Qwen, SGLang, Modal, and Zero-Copy Multi-Agent Architecture.